

- 1) `len(list)` → 1
- 2) `sum(list)` → 10 20 30 40 50 [↑] string
- 3) `max(list)` → 50
- 4) `min(list)` → 10

- List Comprehension

↳ 10 20 30 40 50 list 10 20 30 40 50 append

Syntax: `new_list = [x for x in range(5)]`
`new_tuple = (x for x in range(5))`

`new_list = []` # ()

`for x in range(5):`
`new_list.append(x)`

`print(new_list)`

Output

`[0, 1, 2, 3, 4]`

Ex1

`my_list = [10, 20, 30, 40]`
~~10~~ 20 → 40 → 60 → 80

`new_list = [x*2 for x in my_list]` → `[20, 40, 60, 80]`

`print(sum(new_list))` → 200

`y = sum([x/2 for x in my_list])`

`print(y)` → `sum(5, 10, 15, 20) → 50`

`[50]`

`str = ['AAA', 'BB', 'C']`

`[6, 4, 2]`

$str = ['AAA', 'BB', 'C']$
 $new_list = [len(str[i]) * 2 \text{ for } i \text{ in range(len(str))}]$ [6, 4, 2]
- 3 * 2
- 2 * 2
- 1 * 2
new_list = []
for i in range(len(str)):
new_list.append(len(str[i]) * 2)

$num_str = ['10', '20', '30']$
 $nums = [int(num) \text{ for } num \text{ in } num_str]$
 $print(nums) \rightarrow [10, 20, 30]$

List comprehension with filter

↳ if condition is used for
 ↳ append

$nums = [x \text{ for } x \text{ in range(5) if } x \% 2 == 0]$

↳ for even x it's true

[0, 2, 4]

EX1 $my_list = [5, 7, -5, -7]$

$numbers = [my_list[i] \text{ for } i \text{ in range(4) if } my_list[i] < 0]$

$[-5 * 2, -7 * 3] \rightarrow [-10, -21]$

$5 < 0$
 $7 < 0$
 $-5 < 0 \checkmark$
 $-7 < 0 \checkmark$

String

String

Syntax: "Hello", 'Hi', "101", '1.23'

↳ string = immutable sequences of characters

EX for x in "Hello": →

```
print(x)
```

string operators: +, *, in

String Methods

1) boolean string:

- isalnum(), isalpha()
- isdigit(), islower()
- isupper(), isspace()

2) wàñu string :

- lower() , upper()
- strip() , lstrip() , rstrip()

3) Answer: `find()`, `replace()`

4) `.split()` → join string on 27
the list was string!

↳ default: `.split()` ← split in 'words'

↳ โจทย์ถามให้ input () → ให้ string แล้วหาความยาว

[illegible]

↳ 40 สามารถ input ได้ ...

Ex1 รับ ตัวเลขมา 4 ตัว ด้วย X แล้วเปลี่ยนเป็น list ของ ตัวเลข!

สมมติ user พิมพ์มา เช่น "10 20 30 40"

[10, 20, 30, 40]

user_input = input("Enter numbers: ")

raw_user = user_input.split()

for i in range(len(raw_user)):

raw_user[i] = float(raw_user[i])

print(raw_user) → [10, 20, 30, 40]

Ex2 รับ ค่า x_1, y_1, x_2, y_2 ที่ border

point = input("Enter point: ").split()

$x_1 = \text{float}(point[0])$

$y_1 = \text{float}(point[1])$

$X = [\text{float}(point[i]) \text{ for } i \text{ in range(4) if } i \% 2 == 0]$

$x_1, x_2 = X$ [10, 30]

Dictionary

Syntax: { key1: value1, key2: value2, key3: value3 }

Access: dict[key] → value

Update/add: dict[new_key] = new_value

Methods

- 1) .keys() → list of key names
- 2) .values() → list of values
- 3) .items() → list of tuples (key, value)

* list of sum(), min(), max()

Ex

```
scores = {'A': 5, 'B': 7, 'C': 9}
total = sum(scores.values())
print(total) → 21
```

* for loop

```
for k, v in scores.items():
    if k == 'A':
        print(v)
```

} 5

- .get(key, default)

↳ if value for key

is not found, it returns default value

Functions

... File and other functions

* 1 function = 1 job

Final 45 minutes

- 10 jobs
- 1 short program (5)
- 3 programs (10)

Create function

```
def function_name (param1, param2):
```

```
    statement  
    statement  
    statement  
    return x, y, z
```

return statement

- function block

- params of functions

+ global variables

* In function, we can use global variables

Usage: - return value of function

```
a, b, c = function_name(10, 20)
```

```
results = function_name(10, 20)
```

Ex: function to input on user

```
def get_input():
```

```
    x = int(input("Enter x:"))
```

```
    return x
```

```
def main():
```

```
    user_input = get_input()
```

```
    print(user_input)
```

```
print('user input')
```

Ex2

function print_a num

```
def print_a(num):
```

```
    print("a" * num)
```

aaaaa

```
def main():
```

```
    print_a(5)
```

```
main()
```

Ex3

function our_mean numbers

```
def our_mean(numbers):
```

```
    return sum(numbers) / len(numbers)
```

```
def main():
```

```
    mu = our_mean([10, 20, 30])
```

```
    print(mu)
```

```
main()
```